AEGISAI ENTERPRISE EDITION

TECHNICAL DOCUMENTATION SUITE

Installation & Deployment Manual

Self-Contained Deployment Guide for Cloud Run, Docker & Local Runtime

Platform Version:	**2.5.0 Enterprise Edition**
Compliance Scope:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
License:	**Apache License 2.0 (Open Source)**

AegisAI Engineering Team

Confidentiality: Public / Open Source

AEGIS AI ASSURANCE PLATFORM

Installation and Production Deployment Manual

Version: 2.5.0 Enterprise (2026 Edition)

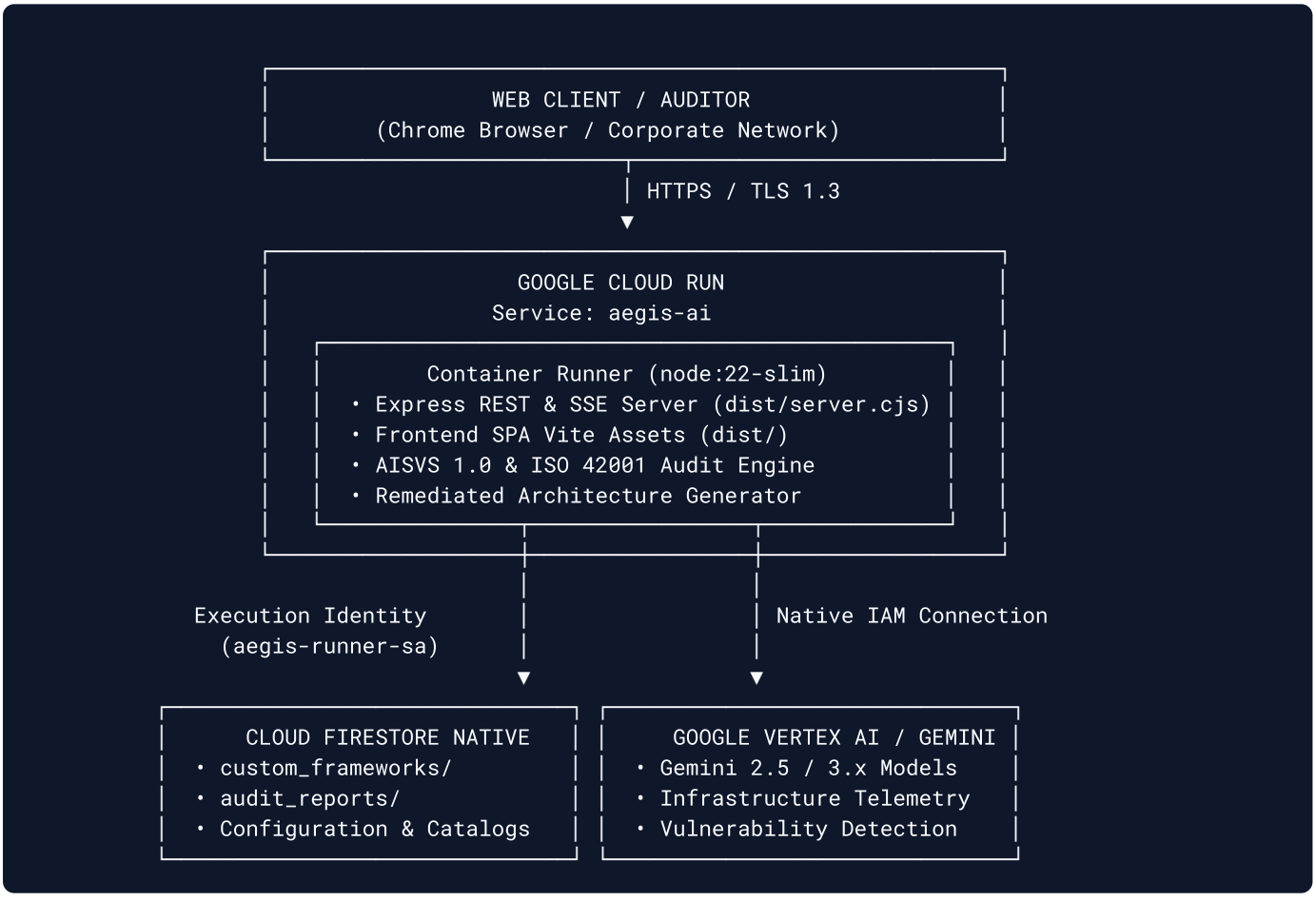
Classification: Technical Infrastructure, Cloud Security, and DevOps Guide

Primary Platforms: Google Cloud Platform (Cloud Run, Firestore Native, Vertex AI, Artifact Registry, Firebase Auth)

Alternative Platforms: Docker Engine, Kubernetes, Linux Bare-Metal/VM Servers

1. Executive Summary and Deployment Architecture

The **AegisAI** platform is designed under a high-resilience, elastic-scalability **Cloud-Native Serverless** paradigm on Google Cloud Platform (GCP). Its architecture decouples compute from data persistence and identity governance:



2. Prerequisites and Tools

To deploy AegisAI to a corporate Google Cloud project, the engineering team must have the following items:

1. **Google Cloud Project**: An active project with a linked Cloud Billing account.

2. **Required IAM Roles for the Deployer**:
   - `roles/owner` or a combination of `roles/resourcemanager.projectIamAdmin`, `roles/run.admin`, `roles/artifactregistry.admin`, `roles/cloudbuild.builds.editor`, `roles/datastore.owner`.
3. **Google Cloud SDK (`gcloud` CLI)**:
   - Minimum version: `gcloud >= 480.0.0`.
   - It is recommended to execute the deployment from **Google Cloud Shell** (`https://shell.cloud.google.com`), which already has `gcloud`, `docker`, `git`, and `node` pre-configured with native authentication.
4. **Local Tools (Optional, if Cloud Shell is not used)**:
   - `docker` >= 24.0.
   - `node` >= 20.10.0 and `npm` >= 10.0.0.

---

3. Method 1: Automated Quick Deployment (Recommended)

AegisAI includes the orchestration script `deploy_cloud_run.sh`, which automates dependency validation, API enablement, provisioning of Service Accounts, the Firestore database, and container building.

Execution Steps:

1. Open **Google Cloud Shell** or your authenticated terminal in GCP.
2. Clone the repository or navigate to the project folder:

```
git clone https://github.com/tu-organizacion/AegisAI.git
cd AegisAI
```

3. Grant execution permissions to the script:

```
chmod +x deploy_cloud_run.sh
```

4. Run the script by passing your GCP project ID:

```
./deploy_cloud_run.sh YOUR_GCP_PROJECT_ID
```

5. The script will automatically execute the 12 provisioning phases and output the assigned URL:

```
🎉 SUCCESSFUL DEPLOYMENT TO GOOGLE CLOUD RUN!
🔗 Cloud Run Service URL: https://aegis-ai-xxxxx-uc.a.run.app
```

---

4. Method 2: Step-by-Step Manual Deployment (For DevOps and SysAdmins)

If corporate policies require every command to be audited or provisioned via CI/CD pipelines (Terraform, Cloud Build Triggers, Jenkins, or GitHub Actions), follow this sequential procedure:

4.1 Configure Project Context

```
export PROJECT_ID="your-gcp-project-id"
export REGION="us-central1"
export SERVICE_NAME="aegis-ai"
export REPO_NAME="cloud-run-source-deploy"

gcloud config set project "$PROJECT_ID"
```

4.2 Enable the 7 Required Google Cloud APIs

AegisAI interacts with compute, AI, build, and database services. Enable the essential APIs:

```
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  aiplatform.googleapis.com \
  firestore.googleapis.com \
  firebase.googleapis.com \
  identitytoolkit.googleapis.com \
  --project="$PROJECT_ID"
```

4.3 Configure Build Pipeline IAM Permissions

To ensure Cloud Build can store sources and write images to Artifact Registry without permission errors:

```
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
COMPUTE_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"
CLOUDBUILD_SA="${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"

# Assign roles to Compute SA (used by Cloud Build in modern environments)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/storage.admin" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/logging.logWriter" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/artifactregistry.admin" --quiet

# Assign roles to Cloud Build Service Agent
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/storage.admin" --quiet
```

4.4 Create the Least-Privilege Runtime Service Account (`C05.2`)

For Zero-Trust security, the container in Cloud Run **must not run with the default Compute account** and **must never be granted broad project-wide `roles/viewer`, `roles/editor`, or `roles/owner` permissions**. Create a dedicated least-privileged account restricted exclusively to Vertex AI inference and Firestore persistence:

```
SA_NAME="aegis-runner-sa"
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud iam service-accounts create "${SA_NAME}" \
  --description="Least-privilege execution identity for AegisAI Cloud Run" \
  --display-name="AegisAI Runner" \
  --project="$PROJECT_ID"

# Assign ONLY Vertex AI inference and Cloud Firestore data roles (Strict Least Privilege C05.2)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/aiplatform.user" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/datastore.user" --quiet
```

4.5 Provision Cloud Firestore in Native Mode

AegisAI stores custom regulatory catalogs and audit reports in Firestore:

```
# Verify or create the database in Native mode
gcloud firestore databases create \
  --database="aegis-ai-db" \
  --location="nam5" \
  --type="firestore-native" \
  --project="$PROJECT_ID" --quiet || echo "Database already exists."
```

4.6 Configure Firebase Authentication and Google Sign-In

To allow users to log in securely:

1. Go to the [Firebase Console](https://console.firebase.google.com/project/_/authentication/providers).
2. Select your project (`$PROJECT_ID`).
3. Under the **Sign-in method** section, select the **Google** provider:
   - Enable the toggle switch.
   - Select the project's corporate support email.
   - Click **Save**.
4. In the [Firebase Console > Project Settings](https://console.firebase.google.com/project/_/settings/general), under the **Your apps** section, register a Web app ( ) named `AegisAI`.
5. Save the resulting configuration in the `firebase-applet-config.json` file:

```
{
  "projectId": "your-gcp-project-id",
  "appId": "1:XXXXXXXXXX:web:XXXXXXXXXXXX",
  "apiKey": "AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "authDomain": "your-gcp-project-id.firebaseapp.com",
  "firestoreDatabaseId": "aegis-ai-db",
  "storageBucket": "your-gcp-project-id.firebasestorage.app",
  "messagingSenderId": "XXXXXXXXXX"
}
```

4.7 Build the Container Image Without Obsolete Layers

Make sure to build with the `--no-cache` flag to ensure TypeScript and Vite artifacts are up to date:

```
# 1. Create repository in Artifact Registry if it does not exist
gcloud artifacts repositories create "$REPO_NAME" \
  --repository-format=docker \
  --location="$REGION" \
  --description="AegisAI image repository" \
  --project="$PROJECT_ID" --quiet || true

# 2. Build and push the image
IMAGE_URL="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/${SERVICE_NAME}:latest"

docker build --no-cache -t "$IMAGE_URL" .
docker push "$IMAGE_URL"
```

(If Docker is not locally available, you can build directly in the cloud using: `gcloud builds submit --tag "$IMAGE_URL" .`).

4.8 Deploy to Google Cloud Run with Zero-Trust IAP & VPC Egress (`C04.3`, `C05.3`)

Deploy the service behind **Cloud Identity-Aware Proxy** (`--no-allow-unauthenticated` + `--iap`), mount `ADMIN_API_TOKEN` from Secret Manager, and enforce controlled VPC egress (`--vpc-egress private-ranges-only`):

```
gcloud beta run deploy "$SERVICE_NAME" \
  --image "$IMAGE_URL" \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --platform managed \
  --service-account "${SA_EMAIL}" \
  --min-instances 0 \
  --max-instances 10 \
  --memory 1Gi \
  --cpu 1 \
  --timeout 300s \
  --port 8080 \
  --set-env-vars "NODE_ENV=production,PORT=8080,PROJECT_ID=${PROJECT_ID},GOOGLE_CLOUD_LOCATION=${REC
  --set-secrets "ADMIN_API_TOKEN=aegis-admin-token:latest" \
  --vpc-connector "aegis-vpc-connector" \
  --vpc-egress "private-ranges-only" \
  --no-allow-unauthenticated \
  --iap
```

4.9 Register the Authorized Domain in Firebase

Once Cloud Run is deployed, retrieve the assigned URL:

```
SERVICE_URL=$(gcloud run services describe "$SERVICE_NAME" --project "$PROJECT_ID" --region "$REGION
echo "Assigned URL: $SERVICE_URL"
```

1. Go to [Firebase Console > Authentication > Settings > Authorized domains] (https://console.firebase.google.com/project/_/authentication/settings).
2. Click **Add domain**.
3. Paste your Cloud Run service domain (e.g., `aegis-ai-607603788049.us-central1.run.app` or your custom corporate domain `aegisai.yourdomain.com`).

---

5. Multi-Tenant Access Control, Corporate Domain Filtering, and Firestore Security Rules (`C05.4`)

AegisAI enforces defense-in-depth access control across three layers (Cloud IAP edge, Frontend OAuth, and Backend/Firestore cryptographic rules):

- **Configurable Allowed Domains**: You can define allowed corporate domains using `VITE_ALLOWED_DOMAINS` (e.g., `google.com,altostrat.com`).
- **Cryptographic Tenant & Owner Validation (`C05.4`)**:
- On the frontend, `Login.tsx` validates the email domain against `VITE_ALLOWED_DOMAINS`, and `App.tsx` / `ApiConnectionView.tsx` bind `ownerUid`, `ownerEmail`, and `tenantDomain` to every created audit report and API key.
- In the database, `firestore.rules` enforces verified corporate domain membership (`isAllowedDomain()`), strict owner binding on creation (`hasValidOwnershipOnCreate()`), owner immutability on update (`ownershipUnchanged()`), and complete client-side denial on `/system_config/{configId}`:

```
function isAllowedDomain() {
  return request.auth != null
    && request.auth.token.email is string
    && request.auth.token.email_verified == true
    && (
      request.auth.token.email.matches('.*@google\\.com$') ||
      request.auth.token.email.matches('.*@altostrat\\.com$') ||
      request.auth.token.email.matches('.*@coppel\\.com$')
    );
}

function hasValidOwnershipOnCreate(data) {
  return (!('ownerUid' in data) || data.ownerUid == request.auth.uid)
    && (!('ownerEmail' in data) || data.ownerEmail == request.auth.token.email);
}
```

- If a user attempts to log in with an unauthorized domain, their session is immediately terminated and rejected at both the Cloud IAP perimeter and Firestore rules engine.

---

6. Operational Verification and Smoke Tests

Once the deployment is complete, verify that the service is functioning correctly:

1. Health Endpoint Check

```
curl -I "${SERVICE_URL}/api/health"
```

Expected response:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status":"ok"}
```

2. HTTP Security Headers Check

```
curl -I "${SERVICE_URL}"
```

Verify the presence of corporate headers:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `X-XSS-Protection: 1; mode=block`
- `Referrer-Policy: strict-origin-when-cross-origin`

3. Automated Integration Test Suite

From the root of the repository, you can run:

```
npm run test:integration
npm run test:gcp
npm run test:security
```

All tests must complete with an exit code of `0`.

---

7. Forensic Troubleshooting Guide

This section documents the most frequent incidents encountered in enterprise GCP deployments and their step-by-step resolution:

Incident 1: `PERMISSION_DENIED` in Cloud Storage during Cloud Build

- **Symptom:** When running `gcloud run deploy --source .` or `gcloud builds submit`, the terminal displays:

```
ERROR: (gcloud.run.deploy) PERMISSION_DENIED: Build failed because the default service account
could not resolve source: Get "https://storage.googleapis.com/...": Access denied.
```

- **Cause:** The default Compute service account (`{PROJECT_NUMBER}-compute@developer.gserviceaccount.com`) lacks the Storage Admin role on the Cloud Build staging bucket.
- **Solution:**

```
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --role="roles/storage.admin"
```

Incident 2: `Firebase: Error (auth/the-service-is-currently-unavailable.)`

- **Symptom:** When clicking "Sign in with Google", the interface displays a red service-unavailable error.
- **Cause:**

1. The **Identity Toolkit** API (`identitytoolkit.googleapis.com`) is not enabled in the GCP project.
2. The `firebase-applet-config.json` file contains keys or identifiers from an old sandbox project that has been decommissioned.
    - **Solution:**

```
# 1. Enable the API
gcloud services enable identitytoolkit.googleapis.com --project="$PROJECT_ID"

# 2. Regenerate firebase-applet-config.json with the active project's values
npx -y firebase-tools apps:sdkconfig WEB --project="$PROJECT_ID"
```

Incident 3: `auth/unauthorized-domain` (Unregistered Domain)

- **Symptom:** When attempting to authenticate, the following message appears:

```
The domain of this instance is not yet registered in the Authorized Domains of Firebase Authent
```

- **Cause:** The domain generated by Cloud Run (`*.run.app`) has not been added to the OAuth whitelist in Firebase Authentication.
- **Solution:**

1. Copy the Cloud Run domain without the protocol (e.g., `aegis-ai-607603788049.us-central1.run.app`).
2. Go to **Firebase Console > Authentication > Settings > Authorized domains**.
3. Click **Add domain**, paste the value, and save.

Incident 4: `The user-provided container failed to start and listen on PORT=8080`

- **Symptom:** Deployment fails during the `Creating Revision...failed` phase, and container logs report `Container called exit(1)`.
- **Cause:**

1. The production stage of the Dockerfile was missing the instruction to copy `firebase-applet-config.json` or the `/config` folder.
2. Development dependencies (`dotenv/config` or `vite`) were statically imported in the production server.
   - **Solution:**
3. Verify that the `Dockerfile` contains: `COPY --from=builder /app/firebase-applet-config.json ./firebase-applet-config.json`.
4. Rebuild while forcing Docker to bypass old cached layers:

```
docker build --no-cache -t "$IMAGE_URL" .
docker push "$IMAGE_URL"
```

Incident 5: `FAILED_PRECONDITION: One or more users named in the policy do not belong to a permitted customer`

- **Symptom:** When assigning `allUsers` or an external domain, `gcloud` throws an organizational policy violation error.
- **Cause:** In organizations with the *Domain Restricted Sharing* policy (`constraints/iam.allowedPolicyMemberDomains`) enabled, granting permissions to anonymous accounts or accounts outside the organization is blocked.
- **Solution:**

Assign the invoker role to the corporate domain instead of `allUsers`:

```
gcloud run services add-iam-policy-binding "$SERVICE_NAME" \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --member="domain:yourdomain.com" \
  --role="roles/run.invoker"
```

Incident 6: `403 Forbidden` error when invoking `/api/evaluate` via Firebase Hosting

- **Symptom:** The main page loads, but when uploading a file for evaluation, the call to `/api/evaluate` fails with `403 Forbidden: Your client does not have permission to get URL /api/evaluate from this server`.
- **Cause:** Firebase Hosting rewrites to Cloud Run only forward requests to public services (`allUsers`). If Cloud Run is private, Firebase Hosting does not inject OIDC identity tokens, and Cloud IAM denies the request.
- **Solution:**

Access the service directly via the Cloud Run URL (`https://--.a.run.app`) or configure an **External Application Load Balancer with Cloud Run Backend** and corporate SSL certificates.

---

8. Maintenance and Updates

To deploy new versions or updates to regulatory catalogs:

```
# 1. Pull changes from the repository
git pull origin main

# 2. Validate types and run tests
npm run lint
npm run test:integration

# 3. Rebuild and deploy the new revision
./deploy_cloud_run.sh "$PROJECT_ID"
```

Cloud Run will instantly manage traffic migration (**Zero-Downtime Deployment**) without interrupting active auditor sessions.